Untested: An Overlooked Link in the Software Supply Chain

Author: Evan Ottinger, evan@ottingerdigitallabs.com
Advisor: Jonathan Risto

Accepted: March 25, 2026

Abstract

Supply chain security appears to be missing a link. Commercial static analysis and endpoint detection tools do not thoroughly examine test libraries in software repositories. This oversight by the security industry paved the way for the xz-utils backdoor in 2024, when a threat actor hid malware inside a seemingly innocuous binary file in the test suite (The MITRE Corporation, 2024). Despite this discovery, there is limited public research or tooling to address this gap in supply chain threat analysis. This research explores test code as an attack surface and takes a first step toward creating a tool to help analysts detect and mitigate malware lurking in test libraries.

1. Introduction

High-quality software demands robust, repeatable engineering processes that enable teams to deploy code rapidly. The cornerstone of a mature development project is a comprehensive test suite. Such a suite includes unit, integration, and end-to-end tests. Maintaining this suite lets developers quickly iterate, add features, and refactor code confidently, knowing that changes do not introduce issues (Mohan, 2021). Regression tests also ensure that remediated bugs and vulnerabilities do not reappear, which increases confidence in production integrity (Powell & Smalley, n.d.).

Given the clear benefits of test suites, it is no surprise that Continuous Integration and Continuous Delivery (CI/CD) pipelines typically include a stage for running test code. These test stages provide developers with immediate feedback on their code commits and can require code adjustments before acceptance into the repository (Sacolick, 2024). Although developers may complain about failed pipelines, CI/CD benefits organizations that value repository code hygiene.

Nevertheless, test code has a downside. Test suites serve as early warnings for reliability and security issues. Yet test code is just code—often running in sensitive environments, such as developer workstations or cloud infrastructure (LaunchDarkly, 2021). In short, it is an attack surface that the security research community has not fully explored.

This oversight became apparent when CVE-2024-3094 was disclosed in March 2024. Upstream tarballs of xz, starting with version 5.6.0, were discovered to contain malware disguised within a test file in the source code repository (The MITRE Corporation, 2024). In this case, the liblzma build process extracts an object file from the test file. This object file is then used to patch functions in liblzma, resulting in a modified library. The modified library can be used by software linked to liblzma. The threat actors implemented this malicious code as a backdoor in SSH (Leite & Belov, 2024). A Shodan search (Figure 1) showed that, once in production, the affected libraries would have enabled remote access to tens of millions of systems once the affected libraries reached production.

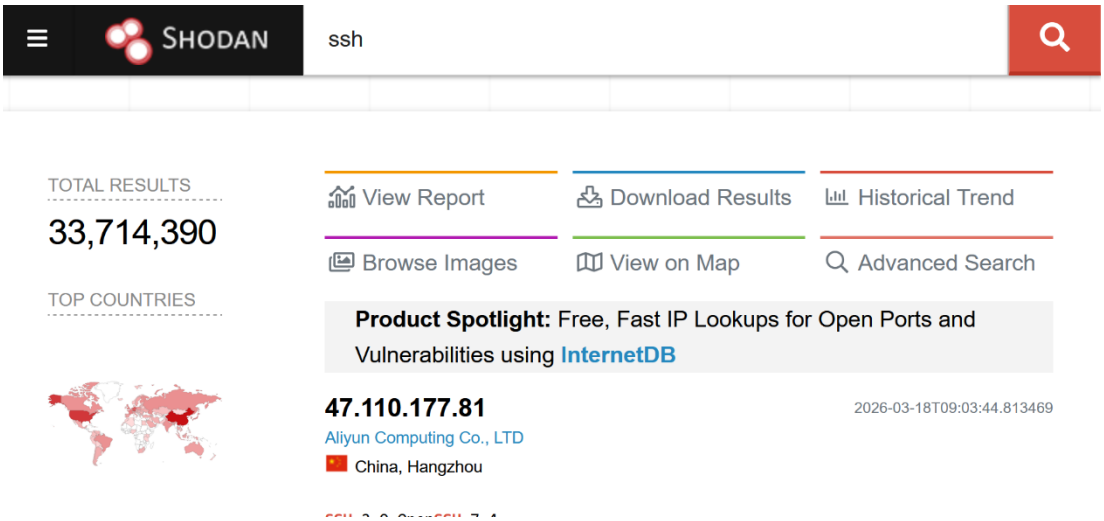


Figure 1: Shodan Search Results

Fortunately, a Microsoft software engineer and PostgreSQL developer, Andres Freund, discovered the backdoor. Freund noticed increased latency in his SSH connections (Freund, 2024). He then discovered that encrypted logins to liblzma were consuming excessive CPU resources. However, performance monitoring tools showed no unusual activity. After some investigation, Freund identified the vulnerability in the upstream xz/liblzma tarballs. The cause was an unused test file containing malware.

CVE-2024-3094 has a CVSS score of 10.0 (The MITRE Corporation, 2024), indicating that a malicious actor with knowledge of the vulnerability can easily exploit it over a remote network to achieve remote code execution (Forum of Incident Response and Security Teams, Inc., 2024). Fortunately, Fruend, acting as an independent researcher out of frustration and curiosity, dampened the malware's impact (Freund, 2024).

Currently, test code is largely overlooked by those outside of the development team at an organization. Software developer and code instructor Mosh Hamedani advises developers to "treat test code like first-class citizens" (Hamedani, 2023). This means giving test code the same attention and care as the production codebase. To prevent supply chain attacks stemming from test libraries, the security community must attend to test libraries as closely as the production codebase at large. The industry cannot rely on volunteer researchers to detect and responsibly disclose such significant threats. In fact, the industry's reliance on a volunteer-maintained library, xz-utils, ultimately led to a

backdoor in downstream projects. This backdoor was introduced via a patient, long-running social engineering campaign (Cox, 2024). To reliably catch future attacks introduced through test libraries, a methodology and supporting tooling should be developed and implemented in organizational security programs.

At the time of this writing, no publicly available research or tools exist to detect and alert against malware embedded in test code. Alarmingly, Poisoned Pipeline Execution is a documented enterprise technique performed by threat actors (The MITRE Corporation, 2025), and test stages comprise a significant aspect of CI/CD pipelines. The objectives of this study are threefold: first, to analyze how test suites may serve as a vector for malware; second, to assess the risks and potential impacts associated with threat actors poisoning test code; and third, to formulate and evaluate a foundational methodology for detection. In pursuit of these aims, the paper investigates the extent to which existing tools can be leveraged to identify malware in test code, and, where such tooling proves inadequate, details the development of a novel tool intended to address this detection gap within supply-chain security.

2. Research Method

2.1. Testing Environment

This research and development focus on Python 3 and the pytest framework. The testing environment is a virtual machine running a default Ubuntu 24.04 LTS installation running in VMWare Workstation Pro 17. Ubuntu was chosen for its popularity and native Python support. Running the operating system in a virtual machine enables portability and it also allows the use of snapshots to revert the system if development causes instability.

The Automated Risk Checker for Test Libraries (arctest) is available on GitHub at https://github.com/evanottinger/arctest and can be cloned using git. Use pipenv to simplify installation in a virtual environment:

1. Install pipenv as described in its official documentation (Pipenv, 2026).
2. After installing pipenv, run `pipenv install` from the project root to set up the virtual environment and install arctest's dependencies.

2.2. Test Plan

The test plan was designed to systematically address the research questions concerning the susceptibility of test suites to malware and the efficacy of detection methodologies. Specifically, each phase of the test plan corresponds to one or more research objectives: implementing and observing malicious behaviors in test code to evaluate attack vectors, assessing the ability of industry-standard tools to detect such threats, and developing and validating a dedicated detection tool. The outlined procedures for writing and executing malicious unit tests, establishing benign controls, and analyzing detection tool outputs were all structured to yield evidence directly relevant to the research aims. In this manner, the test plan not only operationalizes theoretical concerns about test libraries as an attack surface but also empirically connects threat modeling, tool assessment, and detection strategy formulation to the study's core objectives.

2.2.1. Writing Malicious Test Code

The initial challenge of this research was to implement malware within test code. Six unit test files containing malware were implemented using Python's pytest framework (Appendix A). These tests simulate malicious techniques a threat actor may use upon gaining control of a codebase. No special consideration was given when writing the unit tests within the framework. The test functions were implemented according to pytest's documentation; they simply contained malware instead of test code.

tests > test_rev_shell.py > ...

```python
import socket
import os
import pty

# Modified from Python3 #1
# courtesy https://www.revshells.com/


def test_rev_shell():
    RHOST = "192.168.36.131"
    RPORT = 4444

    s = socket.socket()
    s.connect((RHOST, RPORT))
    [os.dup2(s.fileno(), fd) for fd in (0, 1, 2)]
    pty.spawn("sh")

    assert True

```

Figure 2: Reverse Shell Implemented in Pytest

A benign unit test file serves as a control. Successful detection means the library should not alert us to benign tests. The techniques implemented are:

- active data exfiltration
- passive data exfiltration by running an HTTP server
- file mutation
- reverse shell
- sensitive file access
- payload obfuscation

Most of the test code can be run locally without user interaction using the command: `pipenv run pytest`. However, the HTTP server test requires user interaction before proceeding, and the reverse shell requires a listener to be configured before execution. A curl command (Figure 3) and a simple netcat listener (Figure 4) suffice for this research.

```
evan@ubuntu:~/repos/arctest$ curl http://localhost:8080/src/file_to_mutate.py
#!/usr/bin/env python3

num_mutations = 0
print(f"This file has been mutated {num_mutations} times.")
```

Figure 3: Curl Example

```
evan@ubuntu:~/repos/arctest$ nc -nvlp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.36.131 37776
(arctest) \[\e]0;\u@\h: \w\a\]\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]$ whoami
whoami
evan
(arctest) \[\e]0;\u@\h: \w\a\]\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]$ exit
exit
```

Figure 4: Netcat Listener Example

Note that the domains used in the exfiltration test do not currently host any applications. The tests will still proceed without user interaction, but may progress slowly due to timeout delays in the Python requests library, unless the domains are changed to ones with an HTTP listener (locally or on the internet).

Assuming the edge cases mentioned above are satisfied, pytest will run the malware contained in the unit tests when executed. The test framework provides no indication of nefarious activity, as expected. Currently, there is minimal error-handling in these tests. If a test throws an error, the output will be displayed in the pytest output. This can be circumvented by implementing error-handling logic to further decrease the likelihood of detection.

```
(arctest) evan@ubuntu:~/repos/arctest$ pytest
============================================= test session starts ==============================================
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/evan/repos/arctest
plugins: anyio-4.12.1
collected 15 items

tests/test_benign.py .....                                                                            [ 33%]
tests/test_data_exfil.py .....                                                                        [ 66%]
tests/test_http_server.py .                                                                           [ 73%]
tests/test_mutate_file.py .                                                                           [ 80%]
tests/test_obfuscation.py .                                                                           [ 86%]
tests/test_rev_shell.py .                                                                             [ 93%]
tests/test_steal_ssh_keys.py .                                                                        [100%]

============================================== 15 passed in 14.08s ==============================================
```

Figure 5: Pytest Output

2.2.2. Examining Existing Tools

Once the viability of malware techniques in test code was proven, freely available tools were examined for efficacy in detecting the malware. Two well-known code analysis tools, Semgrep and Snyk, were chosen for this stage of research. These tools were chosen because they are professionally developed Static Application Security Testing (SAST) tools with free tiers available. The intent was to examine whether existing SAST tools, traditionally used to analyze source code for potential bugs and vulnerabilities, could detect malware embedded in unit tests with default settings. When tested with the default settings, neither tool returned any results.

After examining the documentation, it was determined that Semgrep ignores test directories by default (Semgrep, 2025). To override the default behavior, create an empty file named `.semgrepignore` at the root of the project. After adding the custom ignore file and repeating the command, Semgrep scanned the test directory and returned a finding in the `test_obfuscation.py` test file. However, the finding highlighted the use of an unsafe Python method, `exec()`. The alert cautioned the user to avoid using `exec()` with external input to avoid command injection vulnerabilities. While this is a valid alert, it overlooks that the payload executed in this test function is base64-encoded, which may be an obfuscation technique. It's nitpicky, but it may be the difference in convincing an analyst to examine the source code or not.

```
(reimagined-carnival) evan@ubuntu:~/repos/reimagined-carnival$ semgrep scan ./tests/

┌─────────────┐
│ Semgrep CLI │
└─────────────┘

⁝ Loading rules from registry...
Scanning 7 files (only git-tracked) with:

✔ Semgrep OSS
  ✔ Basic security coverage for first-party code vulnerabilities.

✔ Semgrep Code (SAST)
  ✔ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

✘ Semgrep Supply Chain (SCA)
  ✘ Find and fix the reachable vulnerabilities in your OSS dependencies.

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:03
```

Figure 6: Semgrep Command

```
1 Code Finding

    tests/test_obfuscation.py
    ❯❯ python.lang.security.audit.exec-detected.exec-detected
          Detected the use of exec(). exec() can be dangerous if used to evaluate dynamic content. If this
          content can be input from outside the program, this may be a code injection vulnerability. Ensure
          evaluated content is not definable by external sources.
          Details: https://sg.run/ndRX

            7┆ exec(payload)


Scan Summary

✅ Scan completed successfully.
 • Findings: 1 (1 blocking)
 • Rules run: 1161
 • Targets scanned: 7
 • Parsed lines: ~100.0%
 • Scan was limited to files tracked by git
 • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 1161 rules on 7 files: 1 finding.
```

Figure 7: Semgrep Results

In contrast, Snyk documentation did not reveal any non-standard settings that would have caused the application to alert on any unit tests (Snyk, 2025). According to the documentation, a similar configuration file, `.snyk`, can be used to specify directories to ignore. After creating an empty `.snyk` file and rerunning the test command, Snyk did not find or report any vulnerabilities. While Snyk does have SAST capabilities, the tool offers a wide range of functions, including identifying supply-chain vulnerabilities in dependencies, containers, and Infrastructure as Code (Snyk, 2026). While that capability has its place in a security pipeline, it is less useful for novel threats hidden within bespoke test code, which is the focus of this research.



Figure 8: Snyk Results

Unfortunately, the research indicates that a detection gap exists in industry supply chain security tooling. Semgrep and Snyk both excel at providing developer-centered SAST tools. The tools integrate easily into the development workflow and provide actionable outputs to help development teams triage vulnerabilities, code styles, and insecure packages—all of which are net-positive for supply chain security. However, test libraries as an attack surface are not considered by either product family, and no configuration settings within the tools readily solve this problem. For a threat actor, this can present opportunities to embed malware in repository test libraries.

2.2.3. Developing Arctest

After examining the available commercial products, it became apparent that a custom solution was necessary. The project was named arctest, the Automated Risk Checker for Test Libraries. Its name conveys the requirements for implementation: it must be an automated tool that can check and categorize risks within test libraries.

Initially, an existing runtime monitor was used to monitor pytest while it was executing the malicious unit tests. The runtime monitor, py-spy, is a sampling profiler for Python programs (Squirro AG, 2025). Written in Rust, py-spy is a wrapper that can monitor a Python program via its process ID or command line argument. It has three subcommands: record, top, and dump. The subcommand “record” can output an SVG

containing an interactive flame graph of the program's execution (shown in Figure 9), while “top” and “dump” show a live, high-level view of the program's function calls and call stacks, respectively.

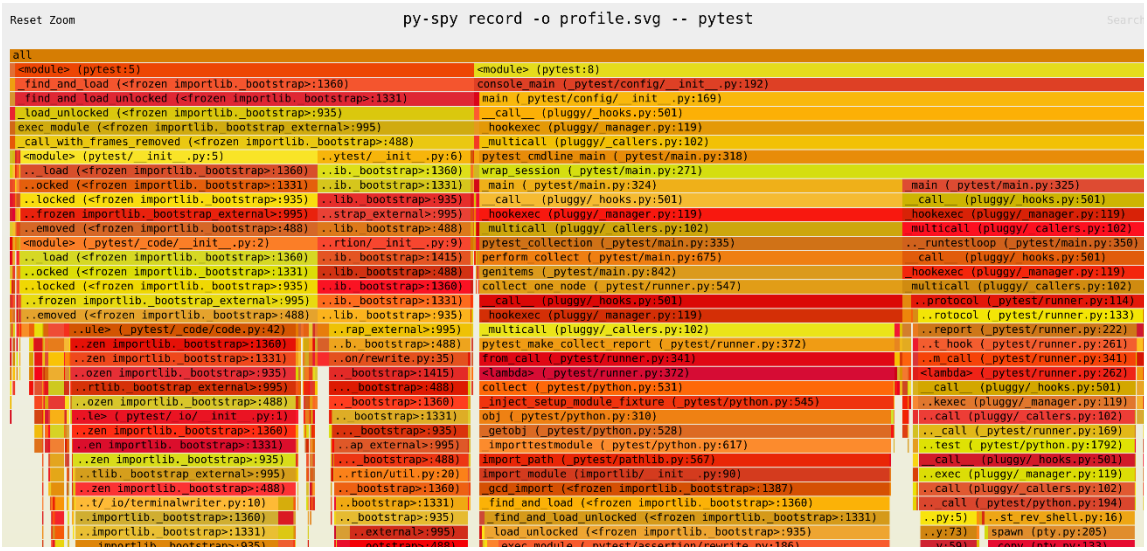


Figure 9: Py-Spy Flame Graph Output

The idea behind using py-spy was to visually examine what pytest function calls the program was spending a relatively large amount of time on, the hypothesis being that blocking methods, such as an HTTP server or reverse shell, would consume visibly more resources than benign system calls. This thought process proved correct, as drilling into the flame graph revealed that function calls within the unit tests were bottlenecks. Upon manual examination, a knowledgeable analyst would likely determine the type of malware executing during the test runtime.

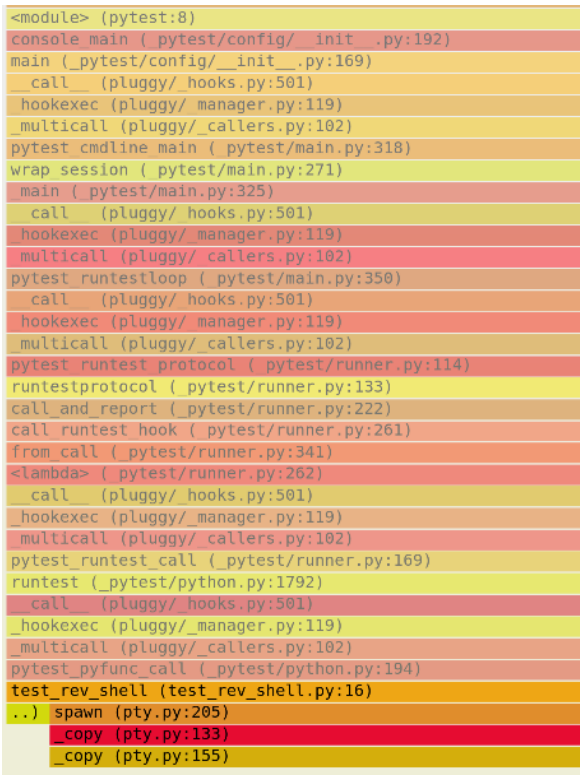


Figure 10: Py-Spy Flame Graph Drill Down

That said, this analysis process is laborious. In practical terms, it would require determining that the test suite was running more slowly than it should. Then the bottlenecked code must be profiled, the profile output examined, and an experienced analyst must decide what is happening in the underlying code based on the output. This analysis pipeline is akin to the discovery of the xz-utils backdoor (Freund, 2024). To achieve the desired functionality for arctest, the analysis and detection process must be automated. Furthermore, the polling process used by py-spy occurs at intervals, which could miss occasional system calls. To ensure higher fidelity in monitoring the test code, runtime instrumentation appeared to be the next logical step.

Since the scope of arctest is limited to test code, development started as a pytest plugin. Pytest plugins can be configured as local `conftest` plugins or installed via the Python package installer, pip (Krekel & Pytest-dev team, n.d.). These plugins are essentially packages that contain one or more hook functions. Hook functions run at specific points in the test life cycle. Commonly, hooks are used to perform “set up” and “tear down” functionality, running tasks before and after a test function.

Likewise, the arctest plugin leverages setup and teardown hooks to monkey-patch security-sensitive functionality within the Python standard library. Like a debugger, these monkey-patched functions are instrumented to provide arctest with insight into their runtime use. In arctest, these instrumentations are referred to as monitors.

At the time of this writing, arctest has four monitors, located in `arctest/runtime_monitors`: `NetworkMonitor`, `FileMonitor`, `ProcessMonitor`, `EnvironMonitor`. Each monitor intercepts identified high-risk functionality and replaces it with a monkey-patched library. For example, ProcessMonitor replaces `pty.spawn()` with a method called `patched_spawn()`. Then, whenever `pty.spawn()` is executed within a test, `patched_spawn()` runs instead. The patched function then creates a finding, which the `InterceptorBase` records. If block mode is enabled, and the finding is categorized as a high or critical, the patched function then raises a `SecurityBlockedError`. Otherwise, it calls the original function and continues execution normally.

The key detection examples currently implemented in arctest's runtime monitors are shell spawning, `stdio` redirection, credential theft, and environment harvesting. Shell spawning and `stdio` redirection, especially when used in combination, indicate a reverse shell being shoveled across a socket connection. Access to sensitive files, such as SSH keys or AWS credentials, may indicate credential theft, especially when combined with network connections. Finally, dumping environment variables may indicate harvesting secrets from the runtime environment, especially when used in conjunction with network connections.

Since the instrumentation provides insight into underlying function calls, the interceptors are triggered even when an attacker implements high-level evasion techniques, such as code obfuscation. Furthermore, when block mode is enabled, the intercepted functions can prevent the malicious code from running when an alert is triggered. Finally, attribution is built in, given that the findings are linked to specific test functions.

However, these runtime detections alone face several problems. First, determined attackers can bypass the heuristic analysis by using `ctypes` or other low-level, unconventional mechanisms. A thorough analysis of threat actor behavior in the wild would be required to identify and analyze detection gaps, then create detections for them. Additionally, monkey-patching adds overhead to test suite execution. In a small, proof-of-concept repository, this effect is minimal, but in a large production codebase, it could add considerable time and expense to test suite execution. Finally, false positives are likely, given that legitimate test infrastructure may trigger findings. False positives in isolation are not necessarily a problem, but in the context of a security program, they can contribute to alert fatigue among analysts and developers and, if excessive, lower acceptance among technical employees within an organization.

During the development of arctest, a Python tool called GuardDog was discovered. GuardDog is "a new open source [sic] project that helps identify malicious Python packages using Semgrep and package metadata analysis" (Wang & Tafani-Dereeper, 2022). Given that GuardDog was implemented in Python using the Semgrep library, it initially seemed that GuardDog would be a useful tool to base the research and development of arctest on. Unfortunately, since GuardDog is tailored to package security, it didn't provide the functionality required to adequately examine test code. The project, however, included a repository of security-focused Semgrep rules that could be used to perform static analysis of test code using the Semgrep Python library.

Semgrep has over 1000 existing Python rules across the community and free editions. Because of this, and the company's commitment to maintaining a free, open-source community edition of its software, it seemed like an appropriate library to use for static analysis.

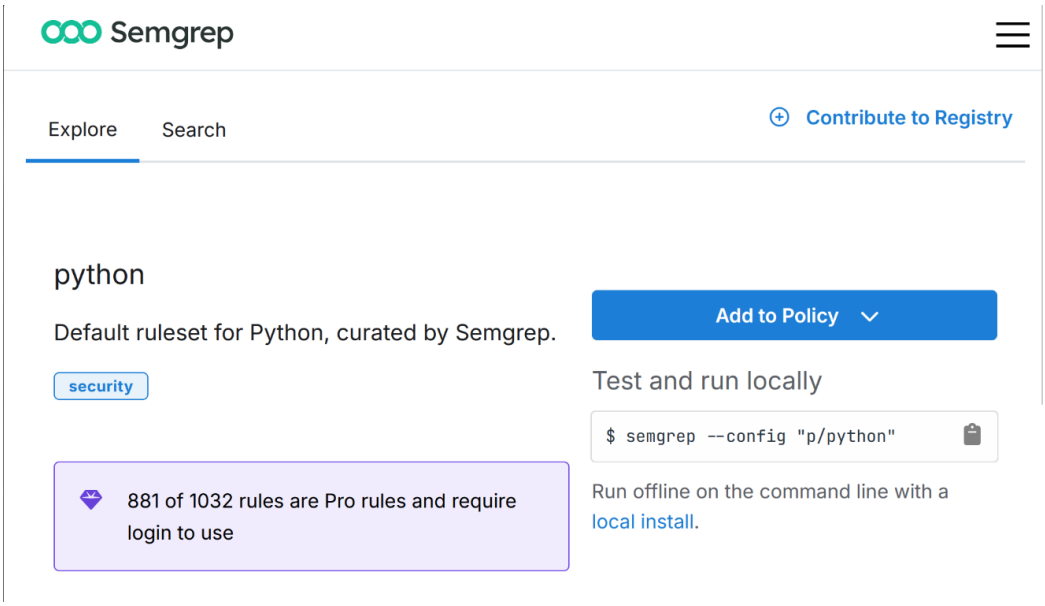


Figure 11: Semgrep Python Rules

However, much of the Python ruleset curated by Semgrep is focused on non-security-related concerns, such as code style. Since each rule adds overhead, and this research specifically examines the security of test code, it seemed appropriate to pare down the rules to those tailored to security. Fortunately, the GuardDog project had already accomplished this.

Because of this, GuardDog rules closely related to identifying the tactics covered by the custom unit tests were added to `arctest/rules`. Then, a wrapper for the Semgrep library was written to use these rules against the test code. This resulted in high-fidelity alerts for the tactics implemented in the demo malware test suite. Furthermore, testing these rules with arctest did not yield any false positives when run against the benign tests in the test library.

```
evan@ubuntu:~/repos/arctest$ pipenv run pytest --arctest tests/
============================================= test session starts ==============================================
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/evan/repos/arctest
plugins: anyio-4.12.1
collecting ...
arctest: Analyzing 7 test file(s)...
arctest: Found 16 static analysis finding(s)
collected 15 items

tests/test_benign.py .....                                                                          [ 33%]
tests/test_data_exfil.py .....                                                                      [ 66%]
tests/test_http_server.py .                                                                         [ 73%]
tests/test_mutate_file.py .                                                                         [ 80%]
tests/test_obfuscation.py .                                                                         [ 86%]
tests/test_rev_shell.py .                                                                           [ 93%]
tests/test_steal_ssh_keys.py .                                                                      [100%]

============================================================
 ARCTEST FINDINGS
============================================================

[CRITICAL] 9 finding(s):
  - test_obfuscation.py:7 - Detected base64 decoding followed by code execution. This is a common obfuscation technique used b
y malware.
  - test_obfuscation.py:7 - Detected base64-decoded content being executed. This is a common technique to hide malicious paylo
ads from static analysis.
```

Figure 12: Arctest Plugin Execution

These results were encouraging; however, given the nature of signature-based testing, these are unlikely to identify novel malware in the wild due to the fragility of the signatures. Adversaries of sufficient maturity are likely to bypass signature-based detection by obfuscating their payloads or circumventing detection altogether (Red Siege, 2021). Regardless, with a layered approach to identifying malware in unit tests and successful deployment against the contrived examples in the malware test library, it was time to test arctest against a small sample of real-world repositories.

2.2.4. Testing Arctest in the Wild

The repositories were selected at random using a helper script generated using Claude Code v2.1.12 running Opus 4.5. To generate the helper scripts, Claude was prompted to create a script that would pull a selection of random PyPI repositories, then run arctest against the test suites in those repositories. The initial script performed the requested actions, but arctest failed to identify malware in the real repositories during its first execution. This was likely due to several factors:

- The small sample size (100) was not adequate to represent the population of over 750,000 PyPI packages.
- Not all packages in PyPI ship with unit tests.

- Not all packages could be downloaded.
- Of the packages that contained tests and could be downloaded, they either did not contain malware, or gaps in detection could not identify them.

It's also worth noting that, for the packages pulled from PyPI, runtime monitoring was not used. Only the Semgrep-based analysis engine was used to scan PyPI repositories. This was for myriad reasons, including the need to create a virtual environment for each repository, install dependencies, and run the test suites, which would cost significant resources given the scale of the package index. Fortunately, for the purposes of this research, the Semgrep rules largely achieve parity with the heuristic-based scanning monitors implemented early in the development of arctest, despite the limitations of signature-based malware analysis mentioned earlier.

To improve the results, Claude was prompted to iterate upon the initial helper script until the listed limitations were addressed. The prompts included adding a CLI option to set the number of packages to pull from PyPI, functionality to automatically pull code from the package's associated GitHub repository if available, and the option to skip packages that do not contain any unit tests. The goal with these prompts was to increase the sample size, increase the likelihood of finding test suites associated with packages, and ensure the actual number of tested packages matched the sample size selected when running the script. The output from Claude improved functionality dramatically, yielding positive results for arctest.

At the time of writing, PyPI had 763,072 packages. To achieve a 95% confidence level with a 5% margin of error, the research needed to sample 384 packages. With the support of the helper scripts produced by Claude, arctest met this requirement in under 3 hours on a single thread. This time could likely be improved by implementing multithreading in the helper script, but for the scope of this research, it was acceptable.

Finally, since the repositories are selected at random from the full list of PyPI repositories, the project needed to implement functionality to ingest the results of a scan.

For this, Claude was prompted to create a helper script that would read the JSON output from the PyPI scanner, extract a list of all repositories in the scan, and run arctest against that list. Claude successfully implemented these requirements in `rescan.py`, allowing anyone with the output from an earlier scan to duplicate the results on their own.

3. Findings and Discussion

Using the Claude-generated helper scripts, arctest ran against 1000 random PyPI repositories to exceed the desired 95% confidence with a 5% margin of error. The findings, discussed below, indicate partial success for arctest in identifying and alerting on malware in packages hosted on PyPI.

3.1. Malware in Unit Test Code

The xz-utils backdoor discovered in 2024 by Andres Freund revealed that seemingly innocuous test files could contain malware (Fruend, 2024). This discovery prompted interest in researching test suites as an attack surface for threat actors. Despite the discovery of CVE-2024-3094, the literature review prior to researching this vector revealed that little public research had been conducted on the security of test code.

As a proof-of-concept, this research implemented several documented tactics, including sensitive file access, reverse shells, passive and active data exfiltration, file mutation, and malware obfuscation. These malicious tests demonstrate that it is possible to write tests that contain malware, with no indication to the end user that anything nefarious occurred. Since test code runs in environments with varying degrees of sensitivity and hardening, this vector can provide threat actors with a foothold on developer workstations or on development infrastructure, on-premises or in the cloud.

In a corporate or volunteer development environment, test code receives less attention during code review than production code, primarily because production code provides value to the organization's customers. If tests are passed, they are unlikely to get a second glance from a development team. While there are no publicly documented records of threat actors being discovered lurking within large test libraries, it is likely that if they were embedded in pipelines via this vector, their dwell time would be significantly above

the mean time to detection for threat actors performing network operations due to the lack of tooling designed to detect malicious behavior in test code.

3.2. Insufficient Monitoring

CVE-2024-3094 represented a highly sophisticated attack chain culminating in a backdoor affecting SSH (Freund, 2024). The attack exploited an unused test file that contained an object extracted at build-time and injected into downstream tarballs of xz-utils. At present, arctest cannot detect attacks of such complexity, highlighting a significant limitation in its current design. More broadly, this research demonstrates that even for less complex threats, including those based on well-known and well-documented techniques, detection within test libraries remains challenging. This suggests that both the tool developed here and industry-standard detection methods are limited in their ability to comprehensively identify malicious code embedded in test libraries. Ultimately, these limitations underscore the need for more advanced detection approaches and continued research to address sophisticated and subtle attack vectors in the software supply chain.

Existing tools are not sufficiently tuned to detect malware in test code. Commercial, off-the-shelf supply-chain monitoring tools like Snyk and Semgrep do not analyze test code by default. Semgrep's community edition does not detect malware techniques, even when default settings are overridden to allow scanning test libraries. GuardDog, a free and open-source tool that leverages Semgrep's Python library, is geared toward detecting and alerting on malicious packages but does not consider test code as a vector in supply-chain security.

For the supply chain to be secure, every link must be examined. Currently, the security industry and research community seem to be overlooking a vital link in that chain. Before the industry can hope to detect test attacks with the level of sophistication seen in CVE-2024-3094, it must be able to reliably identify the everyday techniques shown in this writing.

3.3. The Efficacy of Arctest

This research sought to develop a proof-of-concept tool to analyze test code for malware. Simulated malware samples were written as a baseline to test the tool against

common adversarial techniques that could conceivably exist within test code. However, to assess the scanner's efficacy, it must be tested against repositories in the wild.

Since the scanner is written in Python and leverages the pytest framework, the projects sampled are from PyPI. Currently, PyPI has over 763,000 repositories. To achieve a 95% confidence level that the real value of unit tests malware is within a 5% margin of error, a sample size of 384 or more repositories is required. This research has sampled 1000 repositories from PyPI at random, providing 95% confidence that the true value of unit test malware lies within a 3.1% margin of error.

A helper script was generated with Claude to sample these packages and run the custom Semgrep analyzer against them. Not all PyPI packages include unit tests, so, when possible, repositories are pulled from GitHub. If a repository cannot be pulled or does not have unit tests, the helper script skips to another random repository ad nauseam until one is found with test code. This ensures that only repositories with test code are included in the sample.

After running the Semgrep analyzer against 1000 sample packages, the tool generated 33 alerts spanning 12 packages that warranted further examination. While 30 of these alerts were ultimately determined to be false positives, and 1 alert represented a borderline case, only 1 constituted a clear true positive. These results highlight both the limitations and modest successes of the detection methodology: the high number of false positives suggests the current approach may generate considerable noise, yet the identification of a true positive demonstrates that the tool is capable of flagging security-relevant behaviors. Interpreting these findings calls for nuance, as they indicate not only the challenges of effectively distinguishing malicious activity in test code but also the potential for such tools to support analysts in identifying genuine concerns within large-scale software ecosystems.

Of the false positives, each finding was legitimately suspicious behavior within the broad security landscape, but expected behavior for the packages in question. For example, the stem4tob package contained 6 suspicious patterns, but it is a Tor library. Within the context of its intended functionality, the suspicious patterns are legitimately needed to properly test the software.

```
stem4tob (6 findings):
{
  "package_name": "stem4tob",
  "package_version": "latest",
  "github_url": "pypi:stem4tob",
  "test_files_found": 120,
  "total_findings": 6,
  "high_findings": 4,
  "low_findings": 2,
  "findings": [
    {
      "rule_id": "pytest_malware_guard.rules.compile-exec",
      "severity": "high",
      "file_path": ".../test/unit/__init__.py",
      "line_number": 25,
      "code_snippet": "    code = compile(f.read(), path, 'exec')\n    exec(code)",
      "category": "code_execution"
    },
    {
      "rule_id": "pytest_malware_guard.rules.outbound-socket-connect-external",
      "severity": "high",
      "file_path": ".../test/integ/control/controller.py",
      "line_number": 1107,
      "code_snippet": "        s.connect(('www.torproject.org', 443))",
      "category": "network_communication"
    },
    // + 2 more socket connects, 2 DNS lookups
```

Figure 13: Stem4tob Findings

The borderline-positive result found in the komigram package was due to suspicious usage of the Bitly URL shortener. However, the developers left a comment indicating the destination URL and the reason they used a URL shortening service within the test.

```
komigram (1 finding):
{
  "package_name": "komigram",
  "package_version": "latest",
  "github_url": "pypi:komigram",
  "test_files_found": 143,
  "total_findings": 1,
  "low_findings": 1,
  "findings": [
    {
      "rule_id": "pytest_malware_guard.rules.external-http-request",
      "severity": "low",
      "file_path": ".../tests/test_updater.py",
      "line_number": 616,
      "code_snippet": "        return urlopen(req)",
      "category": "network_communication"
    }
  ]
}
```


Figure 14: Komigram Findings

Finally, the true positive result was found in the informixdb-knight package. This package includes unit test code that reads from the sensitive system file `/etc/passwd`. While this isn't technically malware, it is bad security practice. In an organizational environment, an analyst using this tool would have been alerted to the sensitive file-read operation and could have recommended a more secure solution to the development team.

```
informixdb-knight (2 findings):
{
  "package_name": "informixdb-knight",
  "package_version": "latest",
  "github_url": "pypi:informixdb-knight",
  "test_files_found": 4,
  "total_findings": 2,
  "high_findings": 2,
  "findings": [
    {
      "rule_id": "pytest_malware_guard.rules.passwd-file-access",
      "severity": "high",
      "file_path": ".../tests/test.py",
      "line_number": 86,
      "code_snippet": "    with open('/etc/passwd', 'rb') as f:",
      "category": "system_enumeration"
    },
    {
      "rule_id": "pytest_malware_guard.rules.passwd-file-access",
      "severity": "high",
      "file_path": ".../tests/test_for_wsgi.py",
      "line_number": 102,
      "code_snippet": "    with open('/etc/passwd', 'rb') as f:",
      "category": "system_enumeration"
    }
  ]
}
```


Figure 15: Informixdb-Knight Findings

3.4. Discussion

Arctest takes a needed first step toward shining light into the security of unit test code. Previously, no public tools or research existed to bridge this gap in security monitoring. Although this study did not uncover previously unknown malware, it identified 12 repositories among the 1000 samples that warrant further examination. Based on the results, we can infer that between 712 and 788 PyPI packages contain test code with security implications that need to be addressed. This tool also gives analysts a means to begin finding, triaging, and fixing these concerns.

While these findings are not earth-shattering, they nonetheless highlight a significant industry issue: test code is functionally equivalent to any other code in its

capacity to harbor vulnerabilities or malware, yet it is often neglected in supply chain security assessments. Furthermore, arctest remains in its earliest stages of development, and the identification of security concerns at this point is noteworthy. It is also important to acknowledge broader research limitations, including the constrained focus on specific languages and frameworks and the finite sample of test cases considered. There are likely hundreds, if not thousands, of scenarios and techniques not addressed in this study that could reveal additional undetected security threats within test libraries. As such, the security of test libraries merits more comprehensive investigation and continued attention within the research community.

Despite the progress, it's worth noting that most of the alerts were false positives. Unfortunately, false positives are unlikely to ever be eliminated. That said, there is significant room for improvement in this area for arctest. Most of the false positives stemmed from data exfiltration rules triggered by operations such as printing debug statements and making HTTP requests during integration tests. These types of alerts can and should be tuned to account for such operations within an organization's security program. However, in large-scale scans of package repositories, it seems prudent to triage these alerts regardless.

One possible solution for organizations seeking to implement arctest's capabilities would be to tailor rulesets based on the types of tests running. For example, if only unit tests are running, it would make sense to use all the rules to ensure that no individual unit test is making suspicious network connections. On the other hand, if integration or end-to-end test suites are running, it's better to forgo the exfiltration ruleset, as it's likely to trigger false positives, as seen in this study's results. It will primarily be up to the organization to determine how best to approach monitoring its test suites, as every development team operates differently.

4. Recommendations and Implications

4.1. Recommendations for Practice

In its current state, arctest can be used in public and private Python repositories that rely on pytest for unit testing. As a best practice, this tool should be integrated with

the existing CI/CD infrastructure to validate the security of unit test code prior to the test stage of the pipeline. Depending on the target organization's security posture, it can trigger warnings or outright fail the pipeline if a security concern is discovered. Before the first use, an analyst may consider running arctest on a lab machine to perform initial analysis out of band. This allows the analyst to allow-list any code deemed suspicious but necessary for proper software testing. Regardless of the implementation, users should not use artifacts from the test stage in later pipeline stages.

Since arctest has both a CLI interface and a pytest plugin, organizations adopting pytest can improve their developer experience by including the plugin in their development environment. This will allow developers to see feedback in real time as tests run on their workstations. That way, developers are included in the security cycle and are less likely to encounter surprises during CI/CD pipeline execution after pushing code to a repository.

4.2. Implications for Future Research

This research's scope is limited to the Python pytest framework. Furthermore, the population size of packages tested is limited to PyPI. Python is a popular language; Pytest is a popular framework; and PyPI is a popular package index—but other languages, testing frameworks, and package indexes exist. Future researchers should be encouraged to take the lessons learned from this body of work and apply them to other languages, compiled and interpreted, to build a more broadly scoped tool chain for detecting malware and poor security practices in test code.

Furthermore, arctest's detections are limited to Semgrep rules for static analysis and internally developed runtime analysis rules. It would be prudent to adapt arctest to include a range of analysis techniques, such as YARA rules, and to further develop runtime analytics to detect behaviors missed by signature-based detection.

5. Conclusion

This research has demonstrated the sobering reality that test code is simply code and, as such, can contain malware. As a society dependent on information systems in

almost every facet of our world, we had a near miss with CVE-2024-3094. On the micro level, the curiosity of a single human security researcher prevented what could have been a global catastrophe akin to the WannaCry ransomware event in 2017. But, at the macro level, we got lucky.

Supply-chain security has been widely discussed in the industry, especially since the SolarWinds compromise in 2021. Unfortunately, after reviewing public research on the possibility of threat actors leveraging test code to compromise organizations, this threat appears to have been overlooked by the research community and industry at large.

The purpose of this study was to begin correcting this oversight. The techniques implemented in the malware samples are rudimentary, but even these snuck past the market's existing SAST tools. With a little curiosity and gumption, arctest was created to harness existing security signatures and shine its spotbeam onto otherwise overlooked test suites. Will arctest prevent the next CVE-2024-3094? Not in its current state. But if its development can start a conversation about whether there's a missing link in our supply-chain security, then this research was a success. Humanity solves hard problems by iterating on the work of others. Hopefully, this work will inspire a great mind to build the next iteration.

References

Cox, R. (2024, April 3). *Research!rsc: Timeline of the xz open source attack*. research!rsc. https://research.swtch.com/xz-timeline

Forum of Incident Response and Security Teams, Inc. (2024, June 18). *CVSS v4.0 specification document*. FIRST — Forum of Incident Response and Security Teams. Retrieved March 4, 2026, from https://www.first.org/cvss/v4.0/specification-document

Freund, A. (2024, March 29). *OSS-security - backdoor in upstream xz/liblzma leading to ssh server compromise*. Openwall. https://www.openwall.com/lists/oss-security/2024/03/29/4

Hamedani, M. (2023). *React testing mastery: From basics to advanced techniques* [Video]. Code with Mosh. https://codewithmosh.com/p/react-testing-mastery

Krekel, H., & Pytest-dev team. (n.d.). *Writing hook functions*. Pytest Documentation. Retrieved March 16, 2026, from https://docs.pytest.org/en/stable/how-to/writing_hook_functions.html

LaunchDarkly. (2021, April 6). *Test environments 101: Definition, types, and best practices*. https://launchdarkly.com/blog/test-environments-101-definition-types-and-best/

Leite, A., & Belov, S. (2024, June 24). *XZ backdoor behavior inside OpenSSH*. Securelist. https://securelist.com/xz-backdoor-part-3-hooking-ssh/113007/

The MITRE Corporation. (2024, March 29). *Common vulnerabilities and exposures (CVE)*. CVE-2024-3094. https://www.cve.org/CVERecord?id=CVE-2024-3094

The MITRE Corporation. (2025). *Poisoned pipeline execution, technique T1677 - Enterprise*. MITRE ATT&CK®. Retrieved March 18, 2026, from https://attack.mitre.org/techniques/T1677/

Mohan, G. (2021, July 21). *Seven guiding principles in testing*. Thoughtworks. https://www.thoughtworks.com/insights/blog/testing/seven-guiding-principles-testing

Pipenv. (2026). *Installing Pipenv — pipenv 2026.1.0 documentation*. Pipenv: Python Development Workflow for Humans — pipenv 2026.1.0 documentation. https://pipenv.pypa.io/en/latest/installation.html

Powell, P., & Smalley, I. (n.d.). *What is regression testing?* IBM. Retrieved March 3, 2026, from https://www.ibm.com/think/topics/regression-testing

Red Siege. (2021, August 25). *Bypassing Signature-Based AV*. https://redsiege.com/tools-techniques/2021/08/bypass-sig-av/

Sacolick, I. (2024, April 1). *What is CI/CD? Continuous integration and continuous delivery explained*. InfoWorld. https://www.infoworld.com/article/2269266/what-is-cicd-continuous-integration-and-continuous-delivery-explained.html

Semgrep. (2025, August 28). *Semgrepignore v2 | Semgrep*. Semgrep App Security Platform | AI-assisted SAST, SCA and Secrets Detection. Retrieved March 14, 2026, from https://semgrep.dev/docs/semgrepignore-v2-reference

Snyk. (2025). *Exclude directories and files from Snyk code CLI tests*. Snyk User Docs. Retrieved March 13, 2026, from https://docs.snyk.io/developer-tools/snyk-cli/scan-and-maintain-projects-using-the-cli/snyk-cli-for-snyk-code/exclude-directories-and-files-from-snyk-code-cli-tests

Snyk. (2026). *What's Snyk?* Snyk User Docs. Retrieved March 13, 2026, from https://docs.snyk.io/discover-snyk/whats-snyk

Wang, E., & Tafani-Dereeper, C. (2022, November 15). *Finding malicious PyPI packages through static code analysis: Meet GuardDog*. Datadog Security Labs. https://securitylabs.datadoghq.com/articles/guarddog-identify-malicious-pypi-packages/

Appendix A: Arctest Source Code

The source code for arctest is hosted on GitHub (link below). To reproduce the work shown in this research, use the rescan utility located in `scanner/rescan.py` with the JSON output located in `findings/1000-random.json`. The malicious unit test samples are contained in the arctest repository. They are contained in the `tests/` subdirectory.

- https://github.com/evanottinger/arctest